

LegalEase: AI-Powered Legal Document Generator

Project Description

LegalEase leverages state-of-the-art Generative AI to simplify the creation of legal documents by providing customizable, accurate, and editable templates for a wide range of use cases. Users can generate employment contracts, lease agreements, NDAs, and more—all tailored to their specific inputs like involved parties, effective dates, and key terms.

Built with FastAPI and integrated with a powerful AI core, LegalEase ensures users receive professional-grade legal documents that maintain formatting standards, include custom branding (such as logos and fonts), and support features like editable previews and automatic term tables. With secure handling of sensitive user data and seamless document export options (.PDF, .DOCX, .TXT), the platform bridges the gap between accessibility and legal professionalism.

LegalEase empowers entrepreneurs, professionals, and individuals to generate reliable legal content confidently—without needing a legal background.

Project Team / Team Members

- **Nikil Mathew J**
- **Nm id:** 2913AE877CB6E87B4A041A467282B5EB
- **Shatwick Samuel S**
- **Nm id:** 0433156E69ACCF682F94AD349502E8D9
- **Santhosh G**
- **Nm id:** 97A2738E26AB8BD94A948775F2375D5F
- **Sathik Abbas SD**
- **Nm id:** B3F675A325E5EED9A61414D6D9277777

Scenarios

Scenario 1: A startup founder enters details about a new hire and selects "Employment Contract" as the document type. LegalEase generates a comprehensive agreement with clauses for roles, responsibilities, compensation, and confidentiality. The user customizes the final wording, adds their company logo, and downloads the document as a branded PDF.

Scenario 2: A freelancer needs a Non-Disclosure Agreement for a potential client. By specifying the parties, scope of confidentiality, and effective date, they instantly receive a structured and legally sound NDA.

Scenario 3: A landlord uses LegalEase to draft a residential lease agreement. By inputting the property address, tenant details, and lease terms, the platform creates a formatted document with editable sections and a downloadable version in .DOCX format.

Pre-requisites

- FastAPI Framework Knowledge: [FastAPI Documentation](#)
- Google Gemini API Familiarity: [Google Generative AI Documentation](#)
- Streamlit for Frontend: [Streamlit Documentation](#)
- Python Programming Proficiency: [Python Official Docs](#)
- Environment Setup: `pip install fastapi uvicorn streamlit python-docx fpdf Pillow requests google-generativeai python-dotenv`

Project Workflow

Activity 1: Model Selection and System Architecture

1.1 Research and select the Model: Gemini 1.5 Pro model from Google Generative AI for legal document generation.

1.2 Define architecture:

- Frontend: Streamlit for user input and document download.
- Backend: FastAPI (`main.py`, `routes.py`) to handle API requests.
- AI Integration: Gemini 1.5 API used inside `ai_core/gemini_generator.py`.

1.3 Set up development environment with `.env` file containing the Gemini API key.

Activity 2: Core Functionalities Development

2.1 Implement core generation features: Legal document creation, Editable text preview, Format & export as `.txt`, `.docx`, and `.pdf`.

2.2 Build FastAPI backend: Endpoint `/generate` to process POST requests. Accepts `document_type`, `parties`, `terms`, `dates`. Returns generated text from Gemini.

Activity 3: `main.py` & API Logic

3.1 Create and configure `main.py`: Initialize FastAPI app, Route `/` for home, Include router from `routes.py`.

3.2 Handle document generation logic in `routes.py`: Define Pydantic model `DocumentRequest`, Call `GeminiDocumentGenerator.generate_document(...)`.

Activity 4: Frontend with Streamlit

4.1 Design user-friendly Streamlit interface: Input fields for all required data, Responsive layout with title and logo.

4.2 Dynamic document generation preview: Show styled editable output, Option to export as .txt, .docx, or .pdf.

Activity 5: Deployment

5.1 Prepare deployment: Install production-ready dependencies, Set up Procfile, requirements.txt, or Dockerfile if needed.

5.2 Deploy application: Options: Render, Railway, Deta, Streamlit Community Cloud (frontend), Fly.io or VPS for backend.

Activity 6: Conclusion

Finalize and test: Ensure end-to-end functionality, Test Gemini integration under different input combinations.

Milestone 1: Model Selection and Architecture – LegalEase

In this milestone, we focus on selecting the optimal generative AI model for creating personalized and accurate legal documents using Google's Gemini model via the Google Generative AI SDK. We also define the system architecture and set up the development environment to support document generation, editing, and exporting in various formats (.docx, .pdf, .txt).

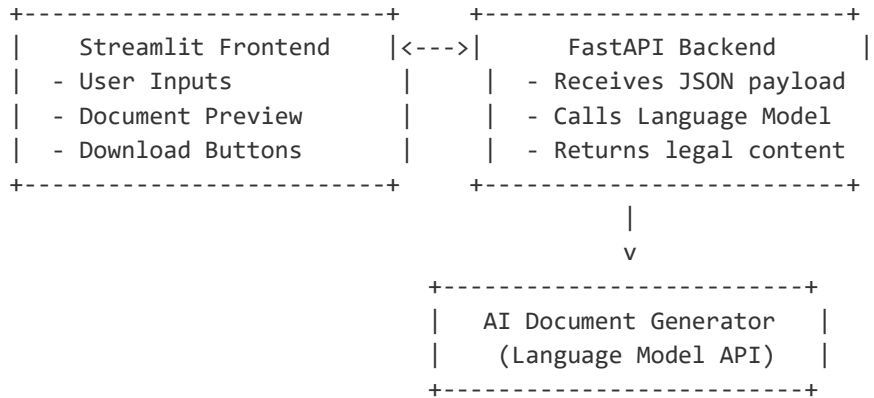
Activity 1.1: Research and Select the Appropriate Generative AI Model

- Understand the Project Requirements: LegalEase must generate structured legal documents (e.g., contracts, NDAs, agreements) based on user inputs such as Document type, Parties involved, Terms and conditions, Effective dates.
- Explore Gemini Model Documentation: Visit Google Generative AI Documentation to evaluate available Gemini models. Review capabilities of gemini-1.5-pro (Long context window, Structured prompting, Fast generation speed).
- Evaluate Model Performance: Benchmarks indicate that gemini-1.5-pro is capable of handling complex formatting, reasoning, and structured text generation, making it suitable for legal content generation.
- Select the Optimal Model: Selected gemini-1.5-pro. Best suited for drafting formal documents with clear sections and legal jargon.

Activity 1.2: Define the Architecture of the Application

1. Architectural Diagram Components:

- Frontend: Streamlit for user interaction (document input, preview, editing, downloads)
- Backend: FastAPI app (routes input to Gemini model and returns generated content)
- AI Core: Gemini integration for content generation
- Formatting Modules: DOCX, PDF, TXT generators



2. Frontend Functionality: Users provide inputs via TextInput, TextArea, Buttons ("Generate Document", "Edit Document", "Download as TXT / DOCX / PDF"), and styled HTML preview.

3. Backend Responsibilities: Expose POST /generate endpoint, accept JSON input, route to GeminiDocumentGenerator, return structured document text.

4. AI Integration Points: GeminiDocumentGenerator forms a structured prompt and sends it to Gemini via generate_content.

Activity 1.3: Set Up the Development Environment

Install Python 3.10+, create virtual environment, install FastAPI, Uvicorn, Streamlit, Google Generative AI SDK, python-docx, fpdf, Pillow, requests, python-dotenv.

Application Structure:

```

LEGALEASE
├── ai_core
│   ├── gemini_generator.py
│   └── generator.py
├── docs
├── frontend
│   └── app.py
├── Image
│   ├── inverseLogo.png
│   └── Logo.png
├── legalEaseAPI
│   ├── main.py
│   └── routes.py
├── venv
├── .env
├── config.py
├── requirements.txt
└── run.sh
  
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the core functionalities

1. Legal Document Generation: Generate legal documents based on user inputs.

```
# routes.py
@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates
    )
    return {"document": response}

# gemini_generator.py
def generate_document(self, document_type, parties, terms, dates):
    prompt = (
        f"Generate a comprehensive legal document titled '{document_type}'\n"
        f"Involved parties: {parties}\n"
        f"Effective Date: {dates}\n"
        f"Terms and conditions: {terms}\n"
        f"Ensure formal legal structure with multiple sections and legal"
        f"clauses."
    )
    response = self.model.generate_content(prompt)
    return response.text
```

2. Document Output Formatting: Convert raw AI text into structured .docx, .pdf, .txt files with formatting, logos, tables, and footers.

3. User Interaction & Recommendations: Accept user preferences through Streamlit interface.

4. Editable Document & Download: Provides editable text area and multi-format export.

Activity 2.2: Implement the FastAPI Backend

Define routes in FastAPI, process user input via Pydantic & Streamlit forms, integrate Gemini API calls.

Milestone 3: API Logic Integration

Integrate backend API logic that powers document generation using Gemini 1.5 Pro model.

Activity 3.1: Create and Configure main.py

```
from fastapi import FastAPI
from legalEaseAPI.routes import router

app = FastAPI(title="LegalEase - AI Legal Document Generator")
```

```

app.include_router(router)

@app.get("/")
def home():
    return {"message": "Welcome to LegalEase AI Legal Document Generator API"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

Activity 3.2: Handle Document Generation Logic in routes.py

```

from fastapi import APIRouter
from pydantic import BaseModel
from ai_core.gemini_generator import GeminiDocumentGenerator

router = APIRouter()
gemini_generator = GeminiDocumentGenerator()

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str

@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates
    )
    return {"document": response}

```

Milestone 4: Frontend Development – LegalEase

Activity 4.1: Designing and Developing the User Interface

Streamlit configuration, centered logo layout, headers, title, and form inputs for document generation.

Activity 4.2: Creating Dynamic Previews and Export Options

Generate document with backend AI, HTML preview rendering, inline text editor, multi-format download buttons (.txt, .docx, .pdf).

Core Functions & Utilities: sanitize_text, format_docx, format_pdf, format_html_preview.

Milestone 5: Deployment

Activity 5.1: Preparing the Application for Local Deployment

Virtual environment setup, installing dependencies, configuring .env for GEMINI_API_KEY.

Activity 5.2: Testing and Verifying Local Deployment

Launching FastAPI backend via Uvicorn, running Streamlit frontend, testing end-to-end functionality.

Exploring LegalEase

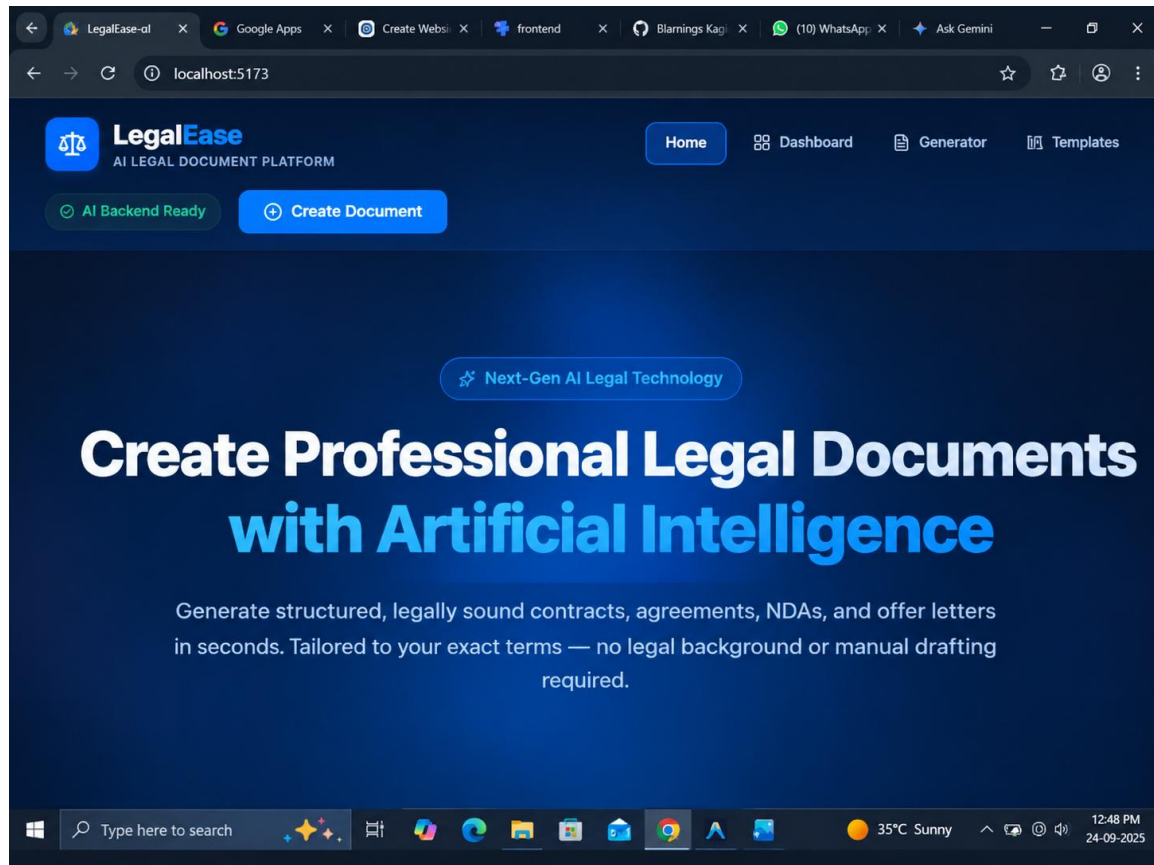


Fig: LegalEase home page — AI-powered legal document generation platform

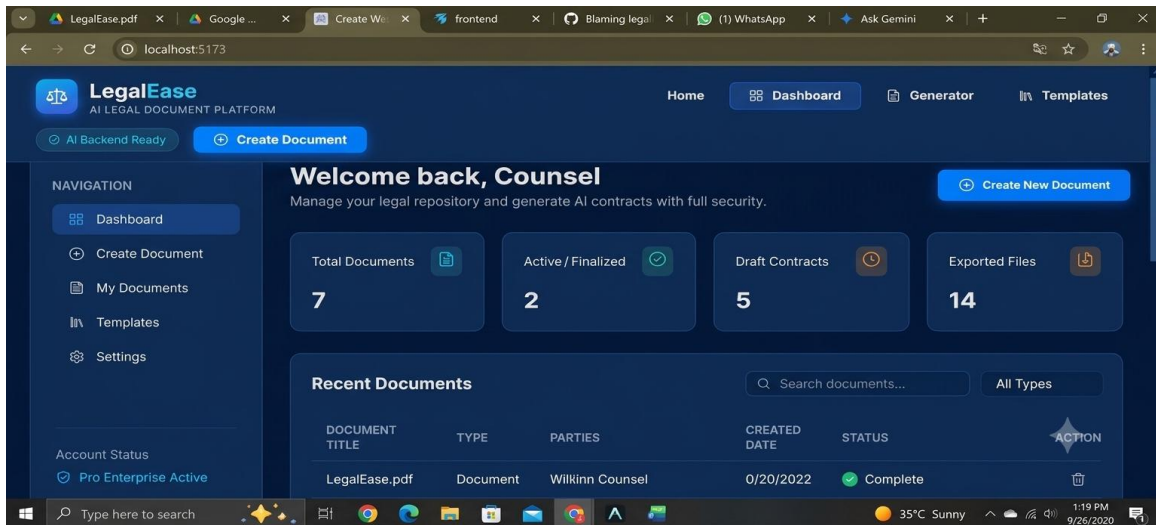


Fig: Dashboard showing total, active, draft, and exported documents

1. Document Type: Agreement, Contract, NDA, Lease Agreement, Employment Offer Letter

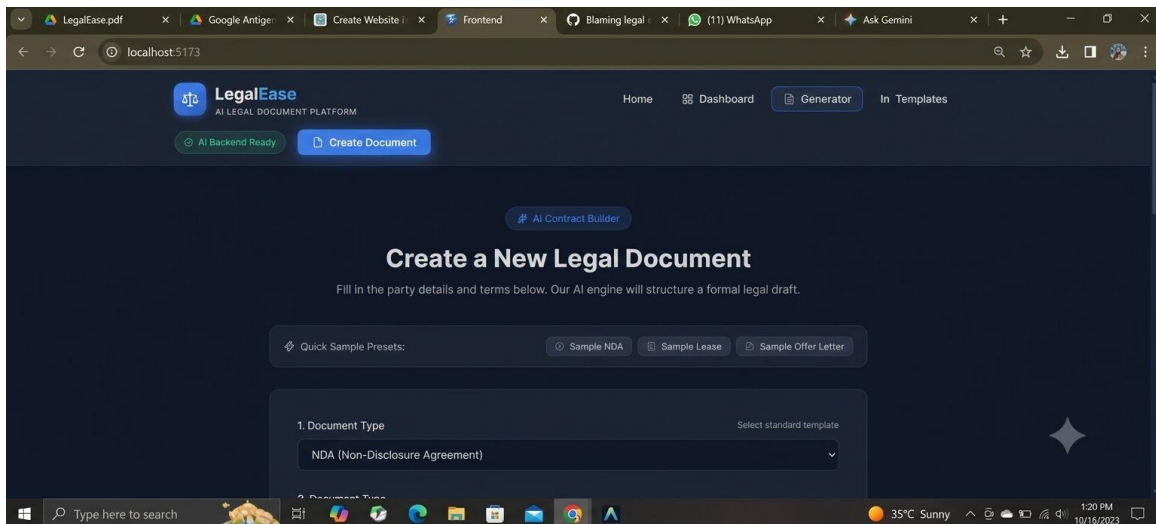


Fig: Generator screen — selecting the Document Type (e.g., NDA)

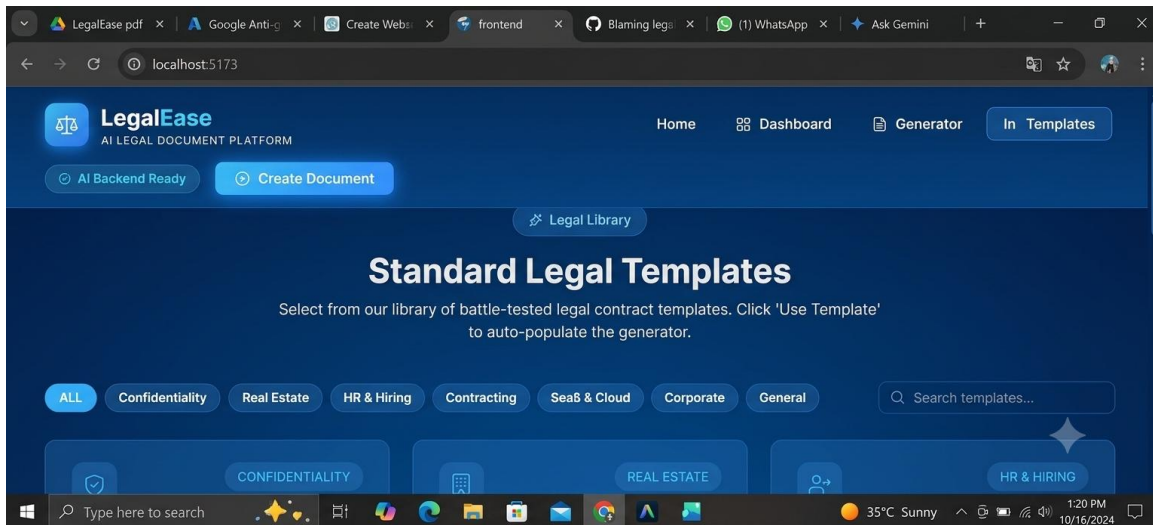


Fig: Standard Legal Templates library for quick document-type selection

2. Parties Involved: John Doe (Freelancer), ABC Corp (Client)
3. Terms & Conditions: Semicolon-separated clauses
4. Effective Date: Date when agreement becomes valid

Conclusion

LegalEase represents a transformative step in bridging the accessibility gap in legal services through the power of artificial intelligence. This project demonstrates how emerging technologies can simplify complex legal language, providing users—regardless of legal background—with a clear, concise understanding of legal documents. By integrating advanced natural language processing (NLP) models with user-centric design, LegalEase empowers individuals to navigate legal content with confidence, thereby promoting inclusivity and informed decision-making.

The platform's ability to analyze and simplify legal text, highlight key clauses, and provide plain-language summaries enhances comprehension and reduces dependency on costly legal consultations for basic understanding. This democratization of legal knowledge is critical in a world where legal literacy remains low, especially among marginalized populations. Furthermore, LegalEase supports multilingual capabilities, ensuring broader reach and utility across different regions and demographics.

From a technical standpoint, LegalEase showcases efficient integration of backend AI models, including document parsing, summarization algorithms, and user-friendly web interfaces. The project's architecture is designed to ensure scalability, data privacy, and real-time responsiveness—key elements for any modern SaaS solution. Through iterative development,

testing, and feedback, the system has been refined for optimal performance and user satisfaction.

In conclusion, LegalEase is more than a technological tool—it is a catalyst for legal empowerment. By fostering transparency, accessibility, and understanding, it upholds the principle that legal information should be a public good, not a privileged resource. Future enhancements could include deeper contract analysis, integration with legal databases, and personalized recommendations. With continued innovation and support, LegalEase has the potential to revolutionize how people interact with the law, making justice more approachable for all.